10 基本資料結構 解答

10-E01 環狀佇列狀態

推導

size=3 的位置從 head=3 起算，有效索引為 3、4、0。enqueue X 寫入 tail=1，tail=(1+1) mod 5=2，size=4。dequeue 使 head=(3+1) mod 5=4、size=3。再 enqueue Y 寫入索引 2，tail=3、size=4。

結論與檢核

最終 head=4、tail=3、size=4，尚未滿。head 與 tail 的數值看似交叉，但 size 明確定義有效區段。

10-E02 sentinel 刪除順序

推導

令 prev=A.prev=N、next=A.next=B。可以依序設 prev.next=next，再設 next.prev=prev。兩個寫入使用事先保存或直接讀到的鄰居，不會再次依賴 A.next。

結論與檢核

刪除後 N.next=B、B.prev=N，且 B.next=N、N.prev=B，剩下一個資料節點 B，並非空串列。若 A 不屬於這份 list，操作可能錯改其他串列，故 membership 是前提。

10-E03 left-child/right-sibling

推導

left[r]=a，sib[a]=b，sib[b]=c。left[a]=d，sib[d]=e。left[d]=f。其餘無子或無兄弟欄位為 NIL。a 的子節點由 x=left[a] 起沿 sib[x] 巡覽，數到 NIL 為止。

結論與檢核

a 的子節點序列為 d、e，共 2 個。sibling 欄位只在同父之下有意義，不能從 e 的 next-sibling 推論其父層。

10-E04 解除連結後才能Free

推導

allocate回1、free變3→4；初始化key/prev/next後，active變0→1→2。先寫next[1]=NIL並更新active tail為1，解除2後，再寫next[2]=3、free=2。終態active0→1，free2→3→4，二集合不交且聯集仍0..4。

10 基本資料結構 解答 （續）

結論與檢核

若active仍有next[1]=2就先改next[2]=free，active將沿2跑入free鏈3→4。Double-free會使free集合重複或成環；free不是將內容清零，而是改變slot所有權與next的語義。

10-E05 右兄弟不是Child

推導

原子樹只包含a及其children遞迴，順序a,d,f,e。Binary式visit(a)會先沿first[a]=d處理d/f/e，再沿sibling[a]=b，接著sib[b]=c，得到a,d,f,e,b,c。

結論與檢核

b,c是a的兄弟，原parent仍r，不能算進a的原樹子樹。要走單一原子樹，應從first[a]列出a的children再逐個遞迴，而不自行走a的sibling。

10-E06 相同Head與Tail的兩種狀態

推導

size=0時空，可enqueue到索引1，不能dequeue；size=4時滿，不能enqueue，可dequeue索引1。兩者都滿足tail=(head+size)mod4。對滿狀態[E,B,C,D]，head1的dequeue回B，再令head2、size3。

結論與檢核

保留一格、只存head/tail版本把head=tail固定解讀為空，滿條件(tail+1)mod4=head，最多容3筆。不能把size版本的4筆容量與保留空槽的判斷混搭。失敗操作須在寫入前檢查，狀態不變。

10 基本資料結構 解答 (續)

原創驗算程式附錄: elementary_checks.py

```
"""Exercise actual boundary operations and compare observable sequences to independent
models."""
from collections import deque
from itertools import product
import random


class Ring:
    def __init__(self,capacity):
        self.data=[None]*capacity
        self.head=0
        self.size=0
    def put(self,value):
        if self.size==len(self.data):
            raise OverflowError
        self.data[(self.head+self.size)%len(self.data)]=value
        self.size+=1
    def get(self):
        if self.size==0:
            raise IndexError
        value=self.data[self.head]
        self.head=(self.head+1)%len(self.data)
        self.size-=1
        return value
    def logical(self):
        return [self.data[(self.head+j)%len(self.data)] for j in range(self.size)]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10 基本資料結構 解答 (續)

```
class Node:
    def __init__(self,key):
        self.key=key
        self.next=self.prev=None


def insert_between(a,x,b):
    x.prev=a
    x.next=b
    a.next=x
    b.prev=x


def unlink(x):
    a,b=x.prev,x.next
    a.next=b
    b.prev=a


def check_list(sentinel,expected):
    values=[]
    node=sentinel.next
    while node is not sentinel:
        assert node.next.prev is node and node.prev.next is node
        values.append(node.key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10 基本資料結構 解答 （續）

```
        assert len(values)<=len(expected)
        node=node.next
    assert values==expected
    assert sentinel.next.prev is sentinel and sentinel.prev.next is sentinel


class Pool:
    def __init__(self,capacity):
        self.next=list(range(1,capacity))+[-1]
        self.prev=[-1]*capacity
        self.used=[False]*capacity
        self.free=0
        self.head=self.tail=-1
    def allocate_before(self,b=-1):
        if self.free==-1:
            raise OverflowError
        x=self.free
        self.free=self.next[x]
        self.used[x]=True
        a=self.tail if b==-1 else self.prev[b]
        self.prev[x]=a
        self.next[x]=b
        if a==-1:
            self.head=x
        else:
            self.next[a]=x
        if b==-1:
            self.tail=x
        else:
            self.prev[b]=x
        return x
    def delete_free(self,x):
        assert self.used[x]
        a,b=self.prev[x],self.next[x]
        if a==-1:
            self.head=b
        else:
            self.next[a]=b
        if b==-1:
            self.tail=a
        else:
            self.prev[b]=a
        self.used[x]=False
        self.next[x]=self.free
        self.free=x
    def validate(self,expected):
        active=[]
        x=self.head
        previous=-1
        while x!=-1:
            assert x not in active and self.used[x] and self.prev[x]==previous
            active.append(x)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

10 基本資料結構 解答 （續）

```
            previous=x
            x=self.next[x]
        assert active==expected and previous==self.tail
        free=[]
        x=self.free
        while x!=-1:
            assert x not in free and not self.used[x]
            free.append(x)
            x=self.next[x]
        assert set(active).isdisjoint(free)
        assert set(active)|set(free)==set(range(len(self.next)))


def main():
    sequences=0
    for length in range(13):
        for operations in product("ED",repeat=length):
            ring=Ring(4)
            reference=deque()
            for step,operation in enumerate(operations):
                before=(ring.data[:],ring.head,ring.size)
                if operation=="E":
                    if len(reference)==4:
                        try:
                            ring.put(step)
                            raise AssertionError("overflow must reject")
                        except OverflowError:
                            assert before==(ring.data,ring.head,ring.size)
                    else:
                        ring.put(step)
                        reference.append(step)
                elif reference:
                    assert ring.get()==reference.popleft()
                else:
                    try:
                        ring.get()
                        raise AssertionError("underflow must reject")
                    except IndexError:
                        assert before==(ring.data,ring.head,ring.size)
                assert ring.logical()==list(reference)
            sequences+=1
    ring=Ring(4)
    ring.put('A');ring.put('B');assert ring.get()=='A'
    for key in "CDE":
        ring.put(key)
    assert ring.data==list("EBCD") and (ring.head,ring.size)==(1,4)
    rng=random.Random(20261024)
    sentinel=Node(None)
    sentinel.next=sentinel.prev=sentinel
    nodes=[]
    for step in range(1000):
        if not nodes or rng.randrange(2):
```

10 基本資料結構 解答 (續)

```
            index=rng.randrange(len(nodes)+1)
            b=sentinel if index==len(nodes) else nodes[index]
            x=Node(step)
            insert_between(b.prev,x,b)
            nodes.insert(index,x)
        else:
            x=nodes.pop(rng.randrange(len(nodes)))
            unlink(x)
        check_list(sentinel,[x.key for x in nodes])
    for x in nodes[:]:
        unlink(x)
        nodes.remove(x)
        check_list(sentinel,[x.key for x in nodes])
    pool=Pool(7)
    active=[]
    for _ in range(2000):
        if not active or (len(active)<7 and rng.randrange(2)):
            active.append(pool.allocate_before())
        else:
            x=active.pop(rng.randrange(len(active)))
            pool.delete_free(x)
        pool.validate(active)
    first={'r':'a','a':'d','d':'f'}
    sibling={'a':'b','b':'c','d':'e'}
    def visit(x):
        result=[x]
        child=first.get(x)
        while child is not None:
            result+=visit(child)
            child=sibling.get(child)
        return result
    assert visit('r')==list('radfebc')
    assert visit('a')==list('adfe')
    trace=Pool(5)
    assert [trace.allocate_before() for _ in range(3)]==[0,1,2]
    trace.delete_free(1)
    trace.validate([0,2])
    assert trace.next==[2,3,-1,4,-1] and trace.free==1
    assert trace.allocate_before(2)==1
    trace.validate([0,1,2])
    assert trace.next==[1,2,-1,4,-1] and trace.free==3
    trace.delete_free(2)
    trace.validate([0,1])
    assert trace.next==[1,-1,3,4,-1] and trace.free==2
    print(f"PASS ring_operation_sequences={sequences} overflow_underflow_preserve_state=True
linked_splices=1000 pool_operations=2000 allocation_partition=True
rooted_subtree_boundary=True")


if __name__=="__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10 基本資料結構 解答 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study